# Domain Specific Accelerator

余嘉閔, 113511060

***Abstract*—I integrate a domain specific accelerator(DSA) to accelerate floating point inner product operation. In this report, I will discuss how I design the DSA and its efficacy compared to the one without it.**

***Keywords—Aquila, FP IP, DSA, ASIC***

## I. Introduction

To enhance the performance of a Convolutional neural network (CNN), I integrate an DSA in Aquila cpu using FP32 format and an FP IP provided by xillinx (with axi stream nonblocking mode).

## II. Discussion of the Design

### *A. Why using DSA*

If we didn't integrate an DSA in the cpu without built-in fpu, the compiler will disassemble the floating point inner product operation into many assembly language instructions, and the result of decomposing the fp operation into integer operation completely by software will come out with a very complex calculation process and will spend too many cpu cycles to complete. Therefore, an DSA is added to do the inner product using specific hardware and FP IP provided by xillinx.(nonblocking)

### *B. How to pass signals between hareware and software*

We will use MMIO to pass the signal. By specifying specific address in sw and dereferencing it, we can control the external hardware by sw.(in risc, every device is regarded as addresses. In this way, we can use purely load/store to control it so we don't have to develop so many instruction)

We can see, in aquila_top.v, the data is passing from cpu_memory stage to the DSA with an arbitor choosing where the data to go by the address in data ports.

The reason why all the data "weight" is fetched from main memory, not micro sd(mmio), is because the cnn program load these data into memory in the beginning of the program.

### *C. Design the DSA software*

After making the fp operation done in fpip, the core bottleneck becomes reading/writing of data to the DSA. Therefore, we need to minimize the time we reading/writing data to it.

I optimized two functions with floating point inner product:

1. conv_3d() in convolution_layer.h
2. fully_connected_layer_forward_propagation() in fully_connected_layer.h

these two functions have similar structures:

```
for(i){ //outer loop
  //do something to a,b
  for(j){ // inner loop
    //inner product operation
    a += b * c
  }
}
```

a[i]: independent of the inner for loop, depending on the outer loop ,and the same 'a[i]' will only be used once, therefore it can be placed outside the inner loop.

b[i,j]: depend on the inner and outer loop, therefore, it can only be kept in the loop.

c[j]: independent of the outer loop, but depends on the inner loop, therefore, we put one inner loop outside the outer loop.

Therefore the revised function is below:

```
//store loop bound of the inner loop to the DSA first
for(j) //inner
  mmio_addr_c[j] = c[j]; //store all the c in DSA register
for(i){
  //do something to a,b
  mmio_addr_a[i] = a[i];  //store a[i] in DSA register
  for(j){
    mmio_addr_b[i,j] = b[i,j];  //store b[i,j] in DSA register
  }
}
```

### *D. Design the DSA hardware*

The DSA has specialized registers for one a[i], one b[i,j], and all the c. (there is only one register to store a and b, but many to store all the c)

Therefore, according to the above revised function, the DSA stores the loop bound of the inner loop,first, and store all the c, and store a[i], and finally store b[i,j].

When one b[i,j] is stored to the DSA, the fp operation will start for once using the fp ip by making the fp_ip_valid bit to be 1 for 1 cycle. The cpu won't stop to wait for the complete of the operation because it has already completed before the coming of the next instruction.

When the fp inner product operation start, the data DSA sent to the ip is: the b[i,j], a[i] stored now and one of c in these c registers. To choose which c[j] to be sent we need a counter, the counter starts at 0 and will +1 when a new b[i,j] is sent , and it will return to 0 when it is already (the loop bound of j) -1. The c to be sent is c[counter].

## III. RESULT

| | The time it took | Compare to the one with DSA |
|---|---|---|
| With DSA | 5314 ms | |
| With FPU | 7915 ms | About 1.49 times |
| Without both | 55118 ms | About 10.4 times |

Their all have the same accuracy, 98%.